Object-Oriented Programming
2301CS201

Darshan UNIVERSITY
योग: कर्मसु कौशलम्

OOP Interview Questions

Prof. Swati R Sharma
Computer Engineering Department
Darshan Institute of Engineering & Technology, Rajkot
swati.sharma@darshan.ac.in

OOP Interview Questions

Explain OOP concepts in detail
(Inheritance, Abstraction, Interface, Polymorphism)

Darshan UNIVERSITY

Inheritance

Application-Oriented Example

Vehicle (Base Class)

- Has Wheels
- Can Move

Car

- Has Doors
- Fuel Type

Motorcycle

- Has Handlebars
- Helmet Required

Truck

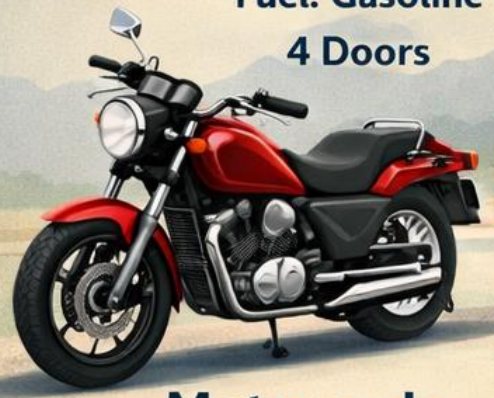
- Has Cargo
- Towing Capacity

The “Vehicle” class serves as the base, while “Car.”, “Motorcycle”, and “Truck” inherit its properties.

Car

Fuel: Gasoline

4 Doors



Helmet Required

2 Wheels

Motorcycle

Cargo Load

Towing

Inheritance

Library Management System Example

LibraryItem (Base Class)

- Title, ISBN
- Borrow()

Book

- Author
- Pages

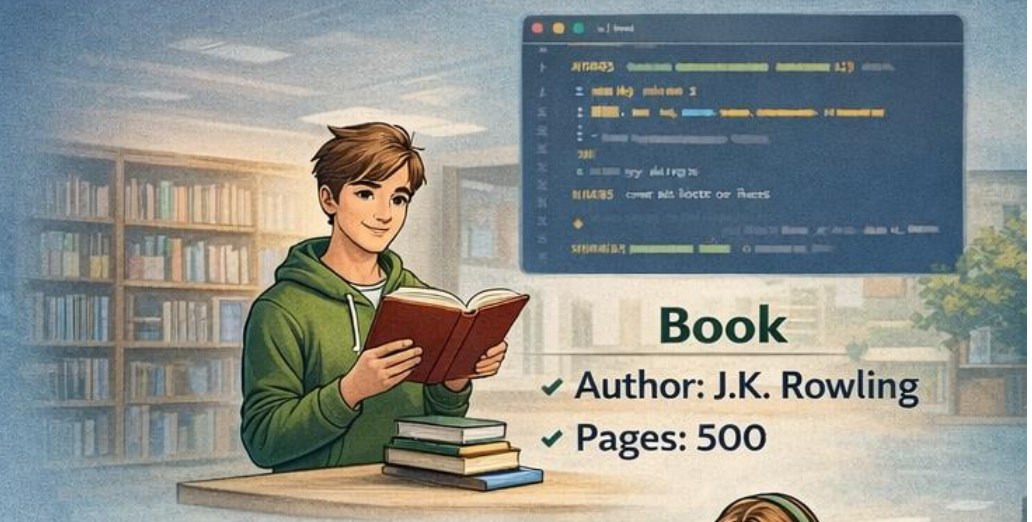
DVD

- Director
- Duration

Magazine

- Issue Number
- Month

The “LibraryItem” class serves as the base, while “Book,” “DVD,” and “Magezine” inherit its properties.



Book

- ✔ Author: J.K. Rowling
- ✔ Pages: 500

DVD

- ✔ Director: Splelberg
- ✔ Duration: 120 min

Magazine

- ✔ Issue: No. 24
- ✔ Month: June

Book

- ✔ Author: J.K. Rowling
- ✔ Pages: 500

Inheritance

Cricket Scoreboard Application Example

CricketPlayer (Base Class)

- Name, Age
- Stats()

Batsman

- Runs
- Batting Avg.

Bowler

- Wickets
- Bowling Avg.

All-Rounder

- Runs, Wickets
- Batting, Bowling Avg.

The "CricketPlayer" class serves as the base, while "**Batsman**," "**Bowler**," and "All-Rounder" inherit its properties.

Inheritance

Cricket Scoreboard Application Example + Types of Matches

CricketMatch (Base Class)

- Teams
- Venue

TestMatch

- 5 Days
- 2 Innings/Game

ODIMatch

- 50 Overs
- 1 Innings/Game

T20Match

- 20 Overs
- 1 Innings/Game

5 Days	50 Overs	20 Overs
2 Innings/Game	1 Innings/Game	1 Innings/Game

The "CricketMatch" class serves as the base, while "TestMatch," "ODIMatch" and "T20Match" inherit its properties.

Inheritance

Exam Management System Example

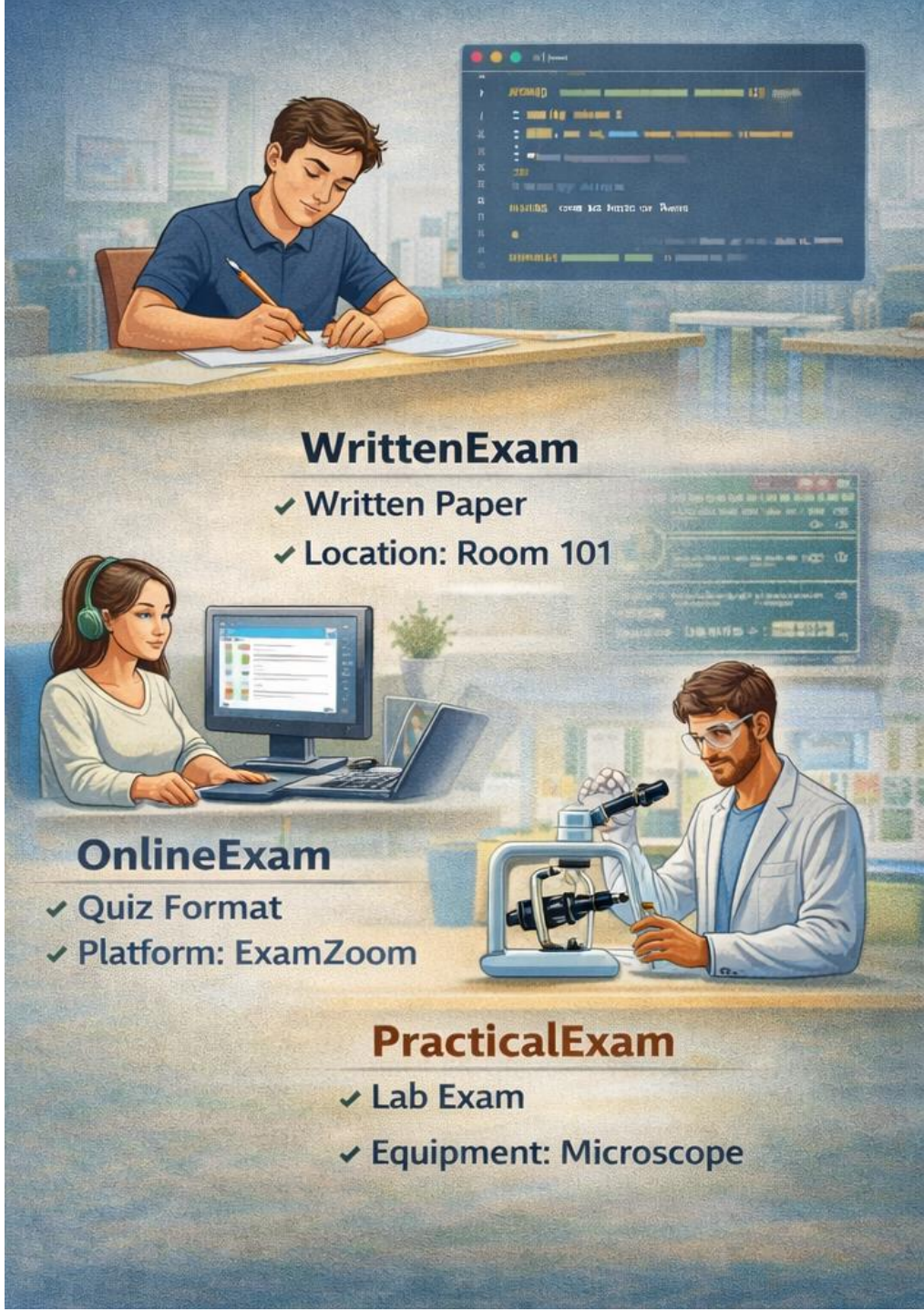
Exam
- Date, Duration
- Conduct()

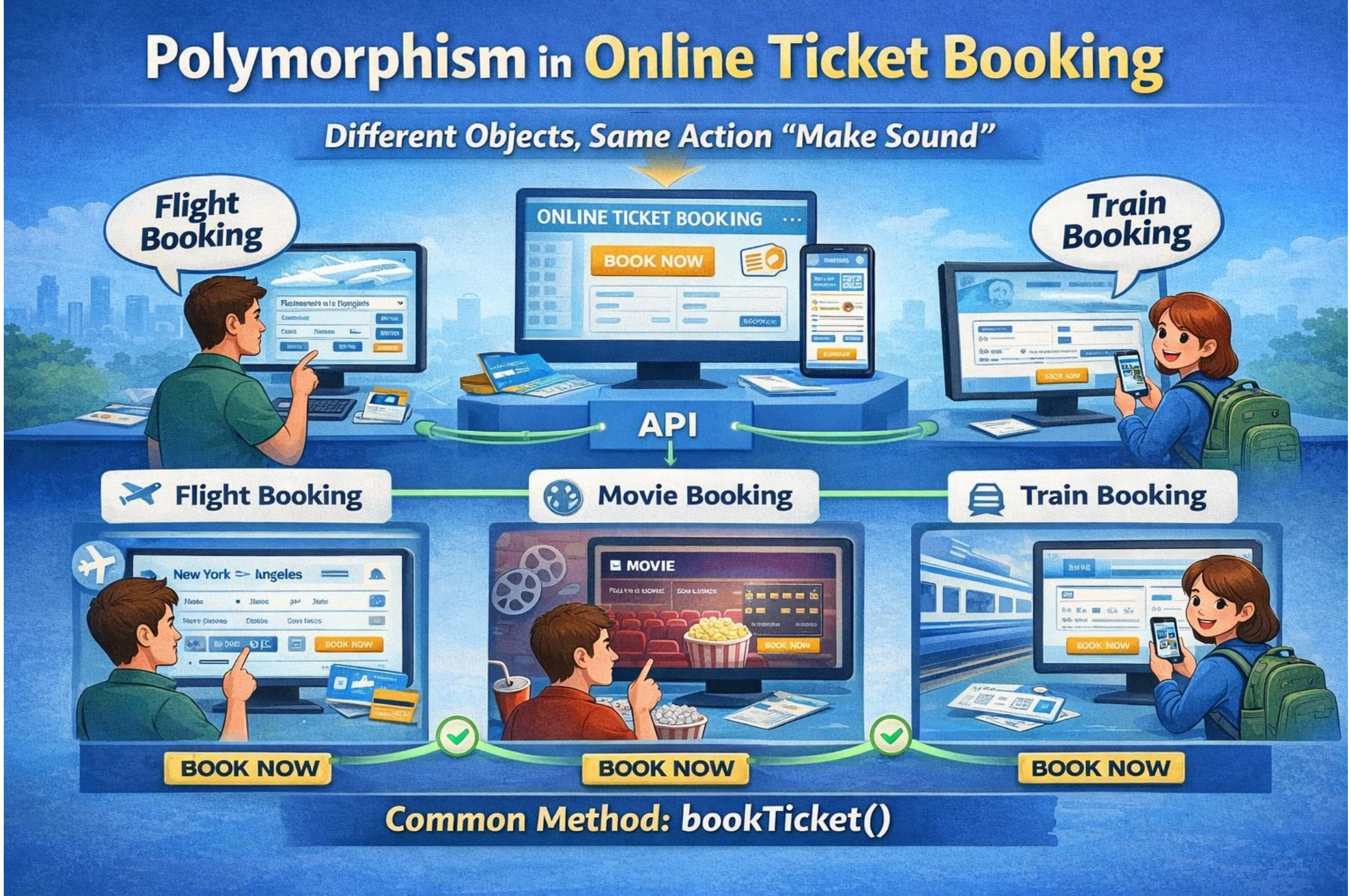
WrittenExam
- Written Paper
- Location

OnlineExam
- Online Platform
- Quiz Format

PracticalExam
- Lab Exam
- Equipment

The “**Exam**” class serves as the base, while “**WrittenExam**”, “**OnlineExam**,” and ‘**PracticalExam**’ inherit its properties.




Polymorphism in Online Ticket Booking
Different Objects, Same Action "Make Sound"
Flight Booking
ONLINE TICKET BOOKING
BOOK NOW
Train Booking
API
Flight Booking
Movie Booking
Train Booking
New York
Angeles
BOOK NOW
MOVIE
BOOK NOW
BOOK NOW
BOOK NOW
Common Method: bookTicket()

Polymorphism in a Delivery System

Different Delivery Methods, Same Action "Deliver Package"

Same Action: deliverPackage()

Encapsulation

Hiding Data & Controlling Access

- Private Data Hidden Inside the Class
- Access via Getters & Setters

Real-World Example: Online Ticket Booking System

- Private Payment Info (Hidden)
- Payment Method chargeCard()
- Confirmation Method sendConfirmation()

Controlled Access to Sensitive Info

Abstraction

OOPJ

HIDES UNIMPORTANT DETAILS

WHAT USERS SEE (IMPORTANT)

- Enter Card Details
- Choose Payment Method
- One-Time Password (OTP)
- Confirm Payment

ABSTRACTION

WHAT IS HIDDEN (UNIMPORTANT)

- Card Information Encryption
- Payment Gateway Processing
- Communication with Bank
- Fraud Detection System
- Payment Status Verification

IMPLEMENTATION DETAILS HIDDEN FROM USER

Abstraction

OOPJ

HIDES UNIMPORTANT DETAILS

WHAT USERS SEE (IMPORTANT)

- Insert Card
- Enter PIN
- Withdraw Cash
- Check Balance
- Deposit Money

ABSTRACTION

WHAT IS HIDDEN (UNIMPORTANT)

- PIN Validation Algorithm
- Communication with Bank Server
- Encryption & Security Checks
- Cash Availability Verification
- Transaction Logging & Database Updates

IMPLEMENTATION DETAILS HIDDEN FROM USER

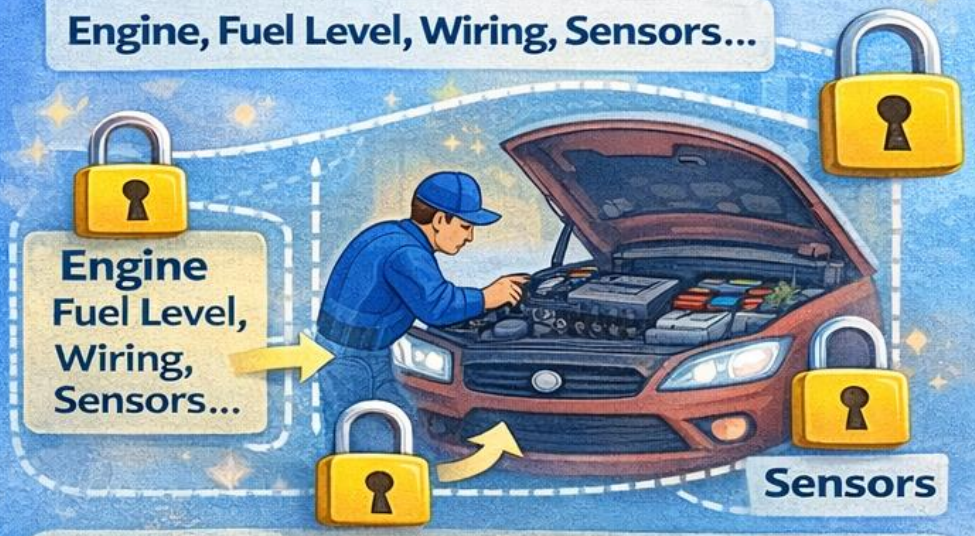
Encapsulation vs Abstraction

Real-World Example: Car

Encapsulation

How it is done?

Engine, Fuel Level, Wiring, Sensors...



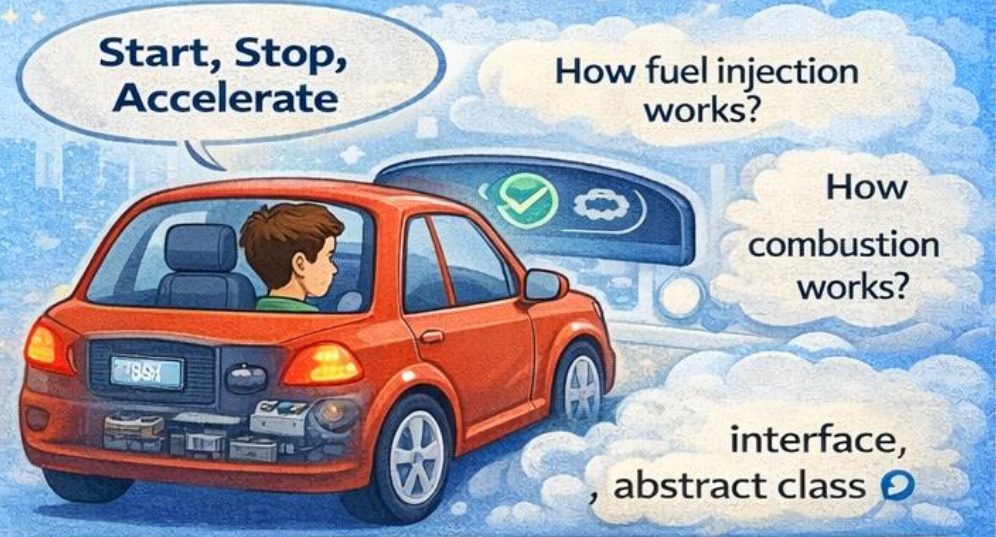

```
class Car {
    private int speed;
    public void accelerate()
}
```

- Hides internal details
- Controls access to data

private, getters/setters

Abstraction

What it does?




```
interface Vehicle:
  void start();
} stop();
```

```
public void start();
public start() {
  Cout. pritln('Car starts');
```

interface, abstract class

Focuses on: Data Protection

private, getters/setters

VS

Abstraction shows WHAT it does

interface, abstract class

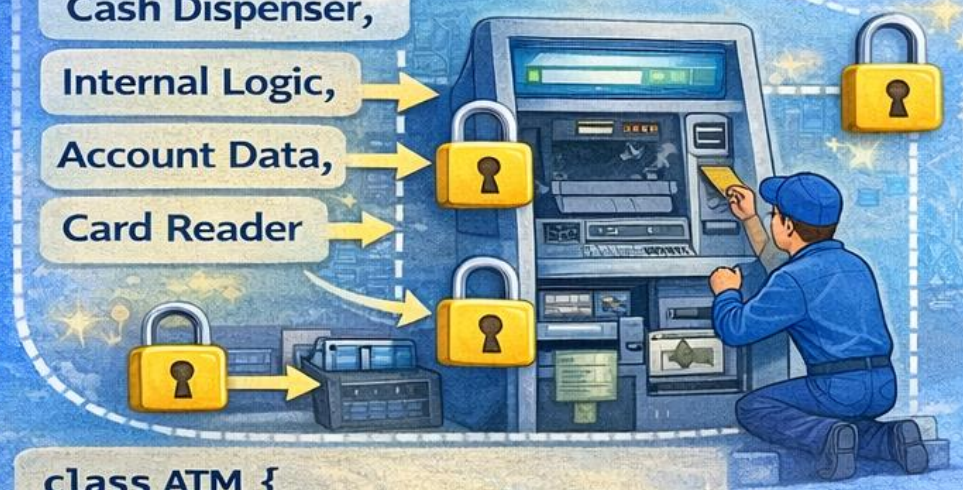
Encapsulation vs Abstraction

Real-World Example: ATM

Encapsulation

How it is done?

Cash Dispenser,
Internal Logic,
Account Data,
Card Reader



```
class ATM {
    private String accountData;
    private void validatePIN(){.../}
}
```

private, getters/setters

Abstraction

What it does?

Withdraw Cash
Check Balance
Transfer Funds

How PIN validiates?
How cash dispenses?

```
interface BankService ;
    void withdrawCash();
    void checkBalance();
    void transferFunds();
}
```

```
public class ATM
implements BankService {
}
```

interface, abstract class

Focuses on: Data Protection
private, getters/setters

VS

Abstraction shows WHAT it does
interface, abstract class

ATM Machine:
Encapsulation vs Abstraction

Encapsulation

Class
- Private Data
- Methods

Sensitive Data Hidden Inside

Hidden (Protected) Data
- Account Balance
- PIN Number
- Card Details
- Transaction Records

Data secure & hidden from direct access

- validatePIN()
- withdrawCash()
- checkBalance()

Encapsulation · ATM Machine

Hides sensitive information, access only via methods

Abstraction

What User Sees
- Withdraw Cash
- Check Balance
- Deposit Money

What User Sees
- Withdraw Cash
- Check Balance

Data secure & from direct access

Sinple interface
- Simple interface
- available options

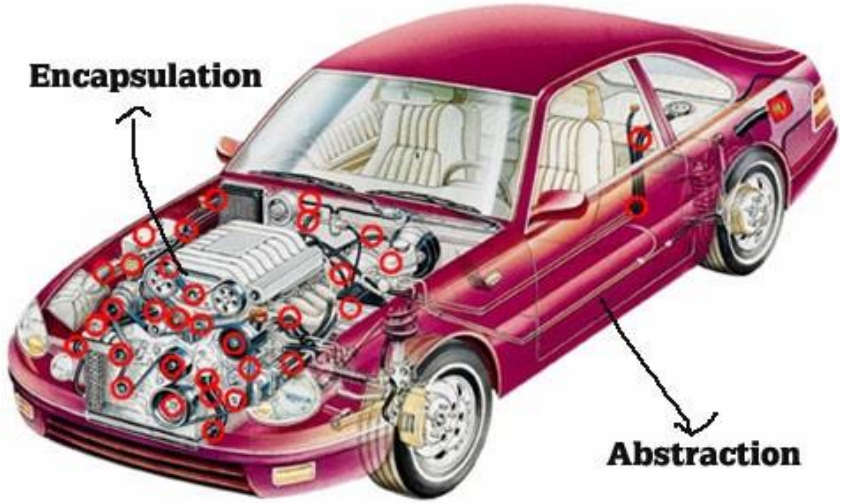
Simple interface with available options

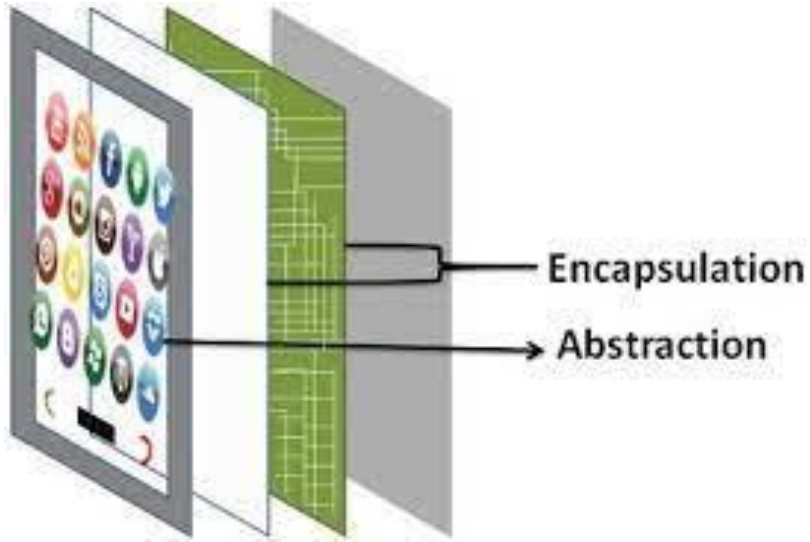
Abstraction · Abstraction

Displays only essential operations, hides internal complexity

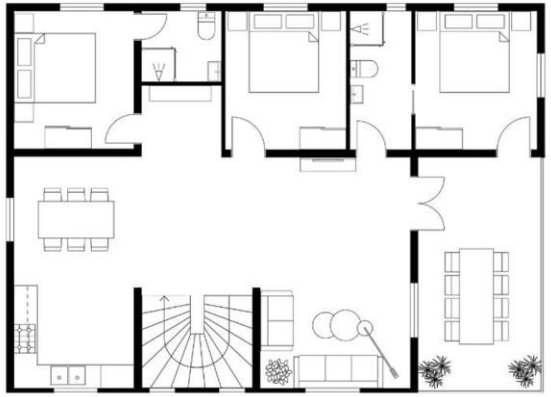
Abstraction vs. Encapsulation

Abstraction	Encapsulation
It means act of removing/ withdrawing something **unnecessary**.	It is act of binding code and data together and keep the **data secure** from outside interference.
Applied at **Designing** stage.	Applied at Implementation stage.
E.g. Interface and Abstract Class	E.g. Access Modifier (public, protected, private)
Purpose: Reduce code complexity	**Purpose**: Data protection





Class: A blueprint of an object

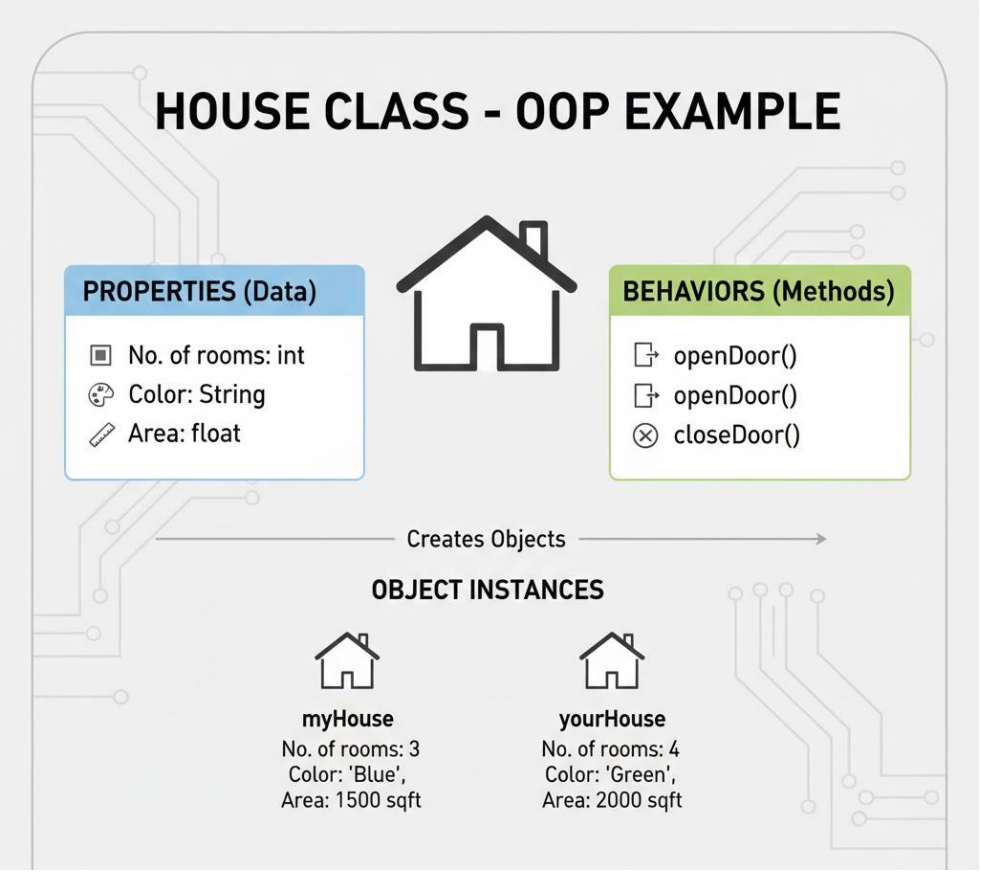


Object: instance of class

Classes and Objects

What is class?

A class is a **blueprint**, **design**, or **template** used to create objects.



When we construct an actual house using this design, that is an **object**.

What is Object?

1. Object is a real **instance** of a class.
2. An object is a **real-world entity** created from a class.
3. **Class** is a blueprint; **object** is the actual thing.

"An **object** is a real-world entity that represents an instance of a class and contains **state, behavior**, and **identity**."

Darshan UNIVERSITY

Class vs. Object

Class	Object
Class is a Blueprint or template.	Object is the instance of a class.
Class creates a logical framework that defines the relationship between its members.	When you declare an object of a class, you are creating an instance of that class.
Class is a logical construct.	An object has physical reality. (i.e., an object occupies space in memory.)
Class is a group or collection of similar object. e.g. Car	An Object is defined as real-world entity e.g.:Audi, Volkswagen, Tesla, Ferrari etc.
Class is declared only once	An Object can be created as many times as required
Class doesn't allocate memory when it is created.	An Object allocates the memory upon creation
Class can exist without its objects.	An Object can't exist without a class.
Class: Profession **Class**: Mobile **Class**: Subject **Class**: Student **Class**: Color	**Object**: Doctor, Teacher, Lawyer, Politician… **Object**: iPhone, Samsumg, 1plus… **Object**: Maths, English, Science, Computer… **Object**: John, Aarav, Smita… **Object:** Blue, Green, Red, Yellow, Violet, Black…

Class vs. Object

Class	Object
Class is a Blueprint or template.	Object is the instance of a class.
Class creates a logical framework that defines the relationship between its members.	When you declare an object of a class, you are creating an instance of that class.
Class is a logical construct.	An object has physical reality. (i.e., an object occupies space in memory.)
Class is a group or collection of similar object. e.g. Car	An Object is defined as real-world entity e.g.:Audi, Volkswagen, Tesla, Ferrari etc.
Class is declared only once	An Object can be created as many times as required
Class doesn't allocate memory when it is created.	An Object allocates the memory upon creation
Class can exist without its objects.	An Object can't exist without a class.
Class: Profession **Class**: Mobile **Class**: Subject **Class**: Student **Class**: Color	**Object**: Doctor, Teacher, Lawyer, Politician… **Object**: iPhone, Samsumg, 1plus… **Object**: Maths, English, Science, Computer… **Object**: John, Aarav, Smita… **Object:** Blue, Green, Red, Yellow, Violet, Black…

OOP Interview Questions

Can Polymorphism Exist Without Inheritance?

- YES — through interfaces.
- Polymorphism means "many forms", and Java supports it via both **inheritance** and **interface** implementation.

```
1.  interface Shape {
2.      double area();
3.  }

4.  class Circle implements Shape {
5.      double r;
6.      Circle(double r){ this.r = r; }
7.      public double area(){ return Math.PI * r * r; }
8.  }

9.  class Square implements Shape {
10.     double s;
11.     Square(double s){ this.s = s; }
12.     public double area(){ return s * s; }
13. }
```

```
14. // Polymorphism — no common parent class!
15. Shape s = new Circle(5);
16. System.out.println(s.area());
```

OOP Interview Questions

Why are constructors not inherited, but methods are?

- Constructors are meant to initialize the specific class they belong to.
- Inheriting them would be dangerous — a Dog constructor shouldn't create a generic Animal.

```
class Animal {
   Animal() { System.out.println("Animal created"); }
   void breathe() { System.out.println("Breathing..."); }
}

class Dog extends Animal {
   Dog() {
      super(); // explicitly call parent constructor
      System.out.println("Dog created");
   }
   // breathe() is inherited — no need to rewrite!
}

Dog d = new Dog();
d.breathe(); // inherited method works fine
```

Methods define behaviour that subclasses naturally share. Constructors define object creation state which is class-specific. You can still call the parent constructor using super()

OOP Interview Questions

If OOP promotes data hiding through encapsulation, why do most applications still need getters and setters? Doesn't that expose the data again?

- Encapsulation doesn't mean "never access data", it means control how data is accessed and modified
- However, blindly generating getters and setters for every field may expose internal state and reduce the benefits of encapsulation. A well-designed class exposes behavior rather than simply exposing its data.
- Encapsulation Means "Hide Implementation", Not "Hide Everything"

OOP Interview Questions

Why is runtime polymorphism considered more powerful than compile-time polymorphism, and are there cases where compile-time polymorphism is actually preferable?

- Runtime polymorphism is often called **more powerful** because it enables **dynamic behavior.**

Feature	Compile-Time Polymorphism	Runtime Polymorphism
Achieved By	Method Overloading	Method Overriding
Binding	Early Binding	Late Binding
Decision Made	During compilation	During execution
Flexibility	Lower	Higher
Performance	Faster	Slightly slower

- **Then Why Do We Need Compile-Time Polymorphism?**

  When you need fast method dispatch with type safety and no inheritance overhead

OOP Interview Questions

Can a constructor be private? Where to use it in a real life ?

- Yes, A private constructor prevents instantiation from outside the class.
- Example

Folowing classes use private constructors to prevent unnecessary object creation.

i. java.lang.Math
ii. java.util.Collections

OOP Interview Questions

If two classes have same name ,same signature but are un-related ,is it polymorphism?

- No. Polymorphism in Java requires a common type (class or interface). Without that, there's no way to treat both objects interchangeably.
- Same method name across unrelated classes is just coincidence. True polymorphism needs a common reference type.

OOP Interview Questions

What is Garbage collection in Java

- Garbage Collection (GC) is Java's automatic memory management.
- The JVM periodically finds and frees objects that are no longer reachable.
- Working: JVM uses a mark-and-sweep algorithm. It marks all reachable objects starting from "GC roots" (local vars, static fields), then sweeps (frees) everything unmarked.
- **Does System.gc() guarantee GC execution?**

  No. It only requests the JVM to perform GC.

System.gc()
↓
Runtime.getRuntime().gc()
↓
JVM receives GC request
↓
JVM decides whether to run GC
↓
If GC runs:
- Mark reachable objects
- Identify unreachable objects
- Reclaim memory

OOP Interview Questions

Can we change return type while overriding ?

- Yes — but only to a more specific (covariant) return type.

- This is called covariant return type. The overriding method's return type must be the same or a subclass of the original return type.

```
1. class Animal {
2.     Animal create() { return new Animal(); }
3. }

4. class Dog extends Animal {
5.     @Override
6.     Dog create() { return new Dog(); } // Dog is a subtype of
   Animal — OK!
7. }

8. // NOT allowed:
9. class Dog extends Animal {
10.    Object create() { return new Object(); } // compile error!
   Object is wider
11. }
```

OOP Interview Questions

- Can we override final method ?
- No, the final keyword on a method explicitly prevents overriding. The compiler throws an error.
- Why use final methods?

  Security-critical logic (authentication, payment) that must not be changed.

Example:

A final class (like String) makes ALL its methods implicitly final.

OOP Interview Questions

What are friend function and friend class?

- Friend functions/classes are a C++ concept — they do NOT exist in Java.
- Java equivalent: Java uses **package-private access** (default access modifier) and **nested/inner classes** instead.

```
// Java approach — inner class accesses outer private fields:
class Outer {
    private int secret = 42;

    class Inner {
        void reveal() { System.out.println(secret); } // OK!
    }
}
```

OOP Interview Questions

What is the virtual function and pure virtual function?

- Again, these are C++ terms, but Java has direct equivalents.
- Virtual function (C++) → any overridable method in Java. In Java, all non-static, non-final, non-private methods are virtual by default.
- Pure virtual function (C++) → abstract method in Java. Forces subclasses to implement it.

OOP Interview Questions

How to achieve abstraction?

It can be achieved using:

- Abstract Class
- Interface

OOP Interview Questions

Can a Java class have multiple constructors?

- Yes! This is called constructor overloading. Each constructor must have a different parameter list.

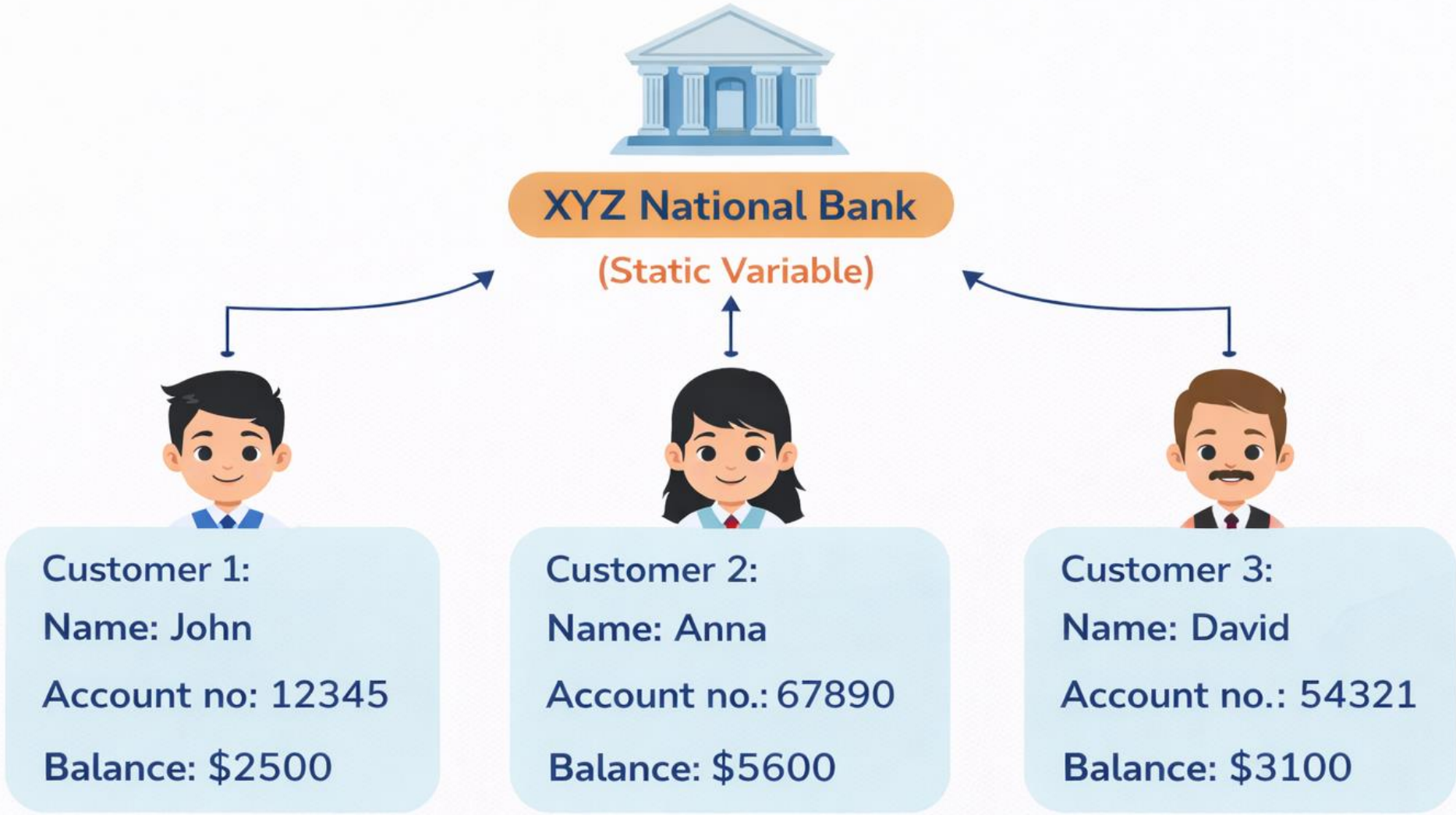
Can we override java constructor?

- No

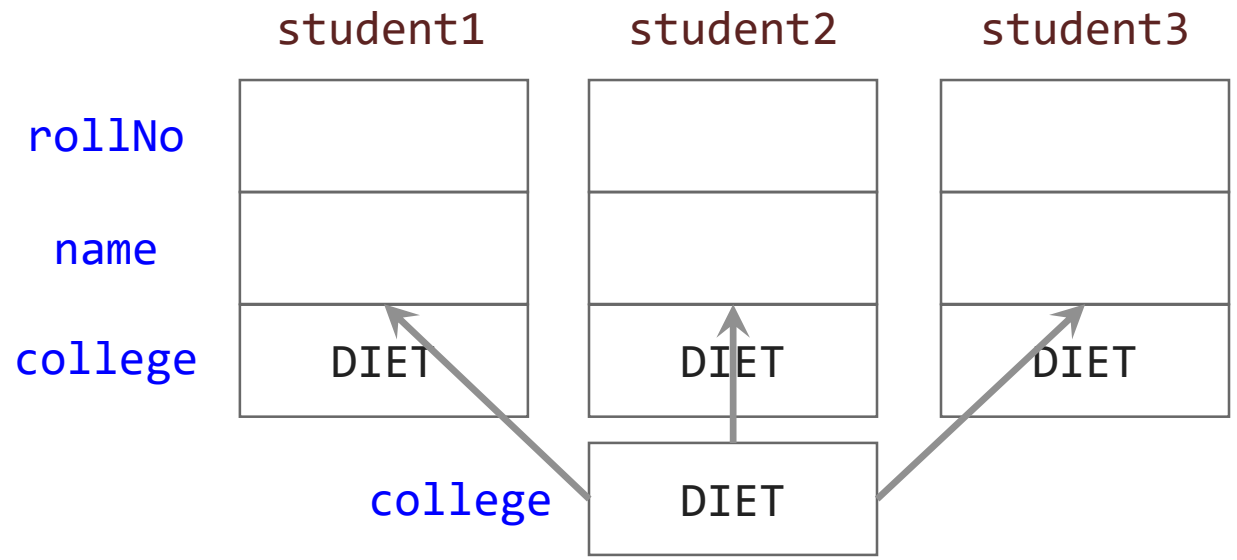
```
1.  class Balance{
2.      int accNo;
3.      double bal;
4.      Balance(){
5.        System.out.println("Darshan University1");
6.        bal=0;
7.      }
8.      Balance(double b){//constructor overload
9.        System.out.println(" Darshan University2");
10.       bal=b;
11.     }
12.     Balance(int a,double b){//constructor overload

13.       System.out.println(" Darshan University3");
14.       bal=b;
15.       accNo=a;
16.     }
17. }
```

Real-World Example of Static Variable

Bank Name in a Customer System



```
class Student {
        int rollNo;
        String name;
        String college="DIET";
        .
        .
        .
}
class StudentDemo {
        public static void main(String args[]) {
                Student student1 = new Student();
                Student student2 = new Student();
                Student student3 = new Student();
                .
                .
                .
        }
}
```



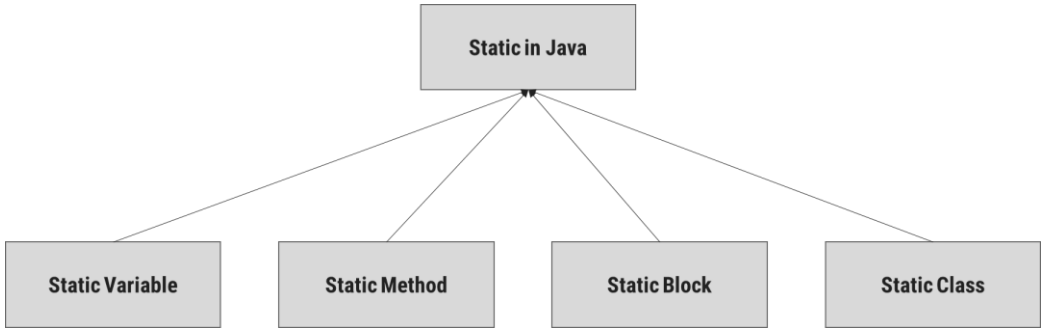
OOP Interview Questions

Can we override a static method in Java?

- No — static methods belong to the class, not the object.
- You can hide them (method hiding), but NOT override them.

Static

1. Can we apply static keyword with a outer class?
2. What is the main use of static keyword in java?
3. How static variable is different from the instance variable?
4. Why is a static method also called a class method?
5. Can we use this or super keyword in static method in Java?
6. Is it possible to overload static methods in a class?
7. Is it possible to override static methods of a class?
8. What is the use of static block in java?
9. Can we declare a static block inside a method?



OOP Interview Questions

What is Functional interface?

- A functional interface has exactly one abstract method.
- It can be implemented via lambda expressions.

```
interface Calculator {
    int operate(int a, int b); // single abstract method
}

// Implement with lambda:
Calculator add = (a, b) -> a + b;
Calculator mul = (a, b) -> a * b;

System.out.println(add.operate(3, 4)); // 7
System.out.println(mul.operate(3, 4)); // 12
```

OOP Interview Questions

What are Generics?

- Generics allow you to write **type-safe, reusable** code where the type is a parameter. Catches type errors at compile-time instead of runtime.

```
// Without generics — unsafe:
List list = new ArrayList();
list.add("hello");
String s = (String) list.get(0); // risky cast

// With generics — type-safe:
List<String> names = new ArrayList<>();
names.add("hello");
String s = names.get(0); // no cast needed, compiler checks it!
```

OOP Interview Questions

What is Serialization?

- Serialization converts a Java object into a byte stream so it can be stored (file, DB) or sent over a network, then reconstructed later (deserialization).
- **Serialization** is the process of converting an object's state into a sequence of bytes so that it can be:
  - Stored in a file
  - Sent over a network
  - Saved in a database
  - Transferred between JVMs

OOP Interview Questions

What is need of Serialization?

```
Student s = new Student(101, "Rahul");
```

- When the program ends, this object is lost from memory. If you want to save it permanently and restore it later, you serialize it.
- A class must implement the **Serializable** interface.

```
import java.io.Serializable;

class Student implements Serializable {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

Serializable is a marker interface, meaning it contains no methods. It simply tells the JVM that objects of this class can be serialized.

Darshan UNIVERSITY

Mutable and Immutable Objects

- The content of mutable object can be changed, while content of immutable objects can not be changed.

```
class MutableClass{
    int a;
    void add5() {
        a = a + 5;
    }
}
public class MutableClassDemo {
    public static void main(String[] args) {
        MutableClass m1 = new MutableClass();
        m1.a = 10;
        m1.add5();
        System.out.println(m1.a);
    }
}
```

```
class ImmutableClass{
    int a;
    int add5() {
        return ( a + 5 );
    }
}
public class MutableClassDemo {
    public static void main(String[] args) {
        ImmutableClass m1 = new ImmutableClass();
        m1.a = 10;
        int ans = m1.add5();
        System.out.println("A in m1 = " + m1.a);
        System.out.println("returned = " + ans);
    }
}
```

Darshan UNIVERSITY

Array of Objects

- We can create an array of object in java.
- Similar to primitive data type array we can also create and use arrays of derived data types (class).

```
class Student{
    int rollNo;
    String name;

    public Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    void printStudentDetail() {
        System.out.println("| "+
        rollNo
        +" | -- | "+
        name +" |");
    }
}
```

```
public class ArrayOfObjectDemo {
    public static void main(String[] args) {
        Student[] stu = new Student[3];

        stu[0] = new Student(101,"darshan");
        stu[1] = new Student(102, "OOP");
        stu[2] = new Student(103,"java");

        stu[0].printStudentDetail();
        stu[1].printStudentDetail();
        stu[2].printStudentDetail();
    }
}
```

Can we do both overloading and overriding?

Yes, a class can use both method overloading and method overriding simultaneously.

OOP Interview Questions

Which type of Inheritance is not supported in java? why?

Java support multiple inheritance? if yes then how to apply it in java?

What is the latest version of java ?

the **latest Java feature release is Java 26 (JDK 26)**, which was released on March 17, 2026.

OOP Interview Questions

How can I apply polymorphism in program ?

When we use method overloading and method overriding in program with example ?

Darshan UNIVERSITY

Difference between abstract class and interface ?

Abstract class	Interface
Abstract class doesn't support **multiple inheritance.**	Interface **supports multiple inheritance**.
Abstract class can **have abstract and non-abstract** methods.	Interface can have **abstract & default** methods.
Abstract class **can have final, non-final, static and non-static variables**.	Interface has **only static and final variables**.
An **abstract class** can extend another Java class and implement multiple Java interfaces.	An **interface** can extend another Java interface only.
A Java **abstract class** can have class members like private, protected, etc.	Members of a Java interface are public by default.

What is collections in java?

Darshan UNIVERSITY

A Collection in Java is a framework that provides a set of classes and interfaces to store, manage, and manipulate groups of objects dynamically.

Example:

ArrayList

LinkedList

Vector

```
1. List<String> a1 = new ArrayList<String>();
2.        a1.add("DU_Honors");
3.         a1.add("DU_MBA");
4.         a1.add("DU_ENGG");

5.         System.out.println("ArrayList Elements");
6.        System.out.print("\t" + a1);

1. List<String> l1 = new LinkedList<String>();
2.        l1.add("DU_Computer");
3.         l1.add("DU_Management");
4. System.out.println("LinkedList Elements");
5.        System.out.print("\t" + l1);
```

Difference between Array vs. ArrayList

Feature	Array	ArrayList
Size	Fixed	Dynamic (resizable)
Package	Built into Java language	java.util package
Data Types	Can store primitives and objects	Stores objects only (uses wrappers for primitives)
Performance	Faster	Slightly slower due to resizing and method calls
Length/Size	length property	size() method
Add/Remove Elements	Manual handling	Built-in methods
Memory Usage	Less overhead	More overhead
Generics Support	No	Yes

Strings are mutable or immutable?

What is immutable ?

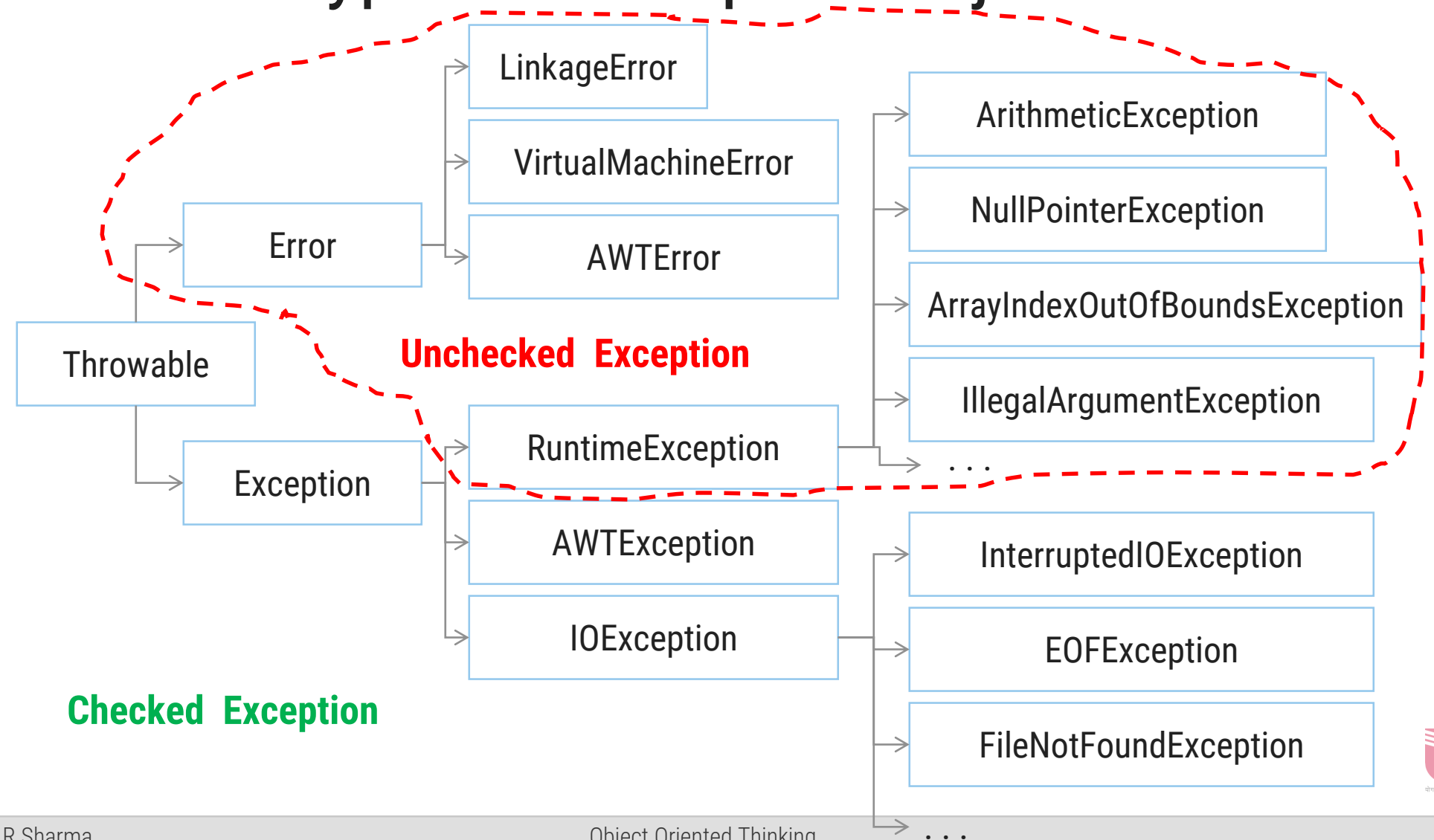
OOP Interview Questions

What is “new“ keyword ?

Difference between Abstraction and Encapsulation

Difference between interface and inheritance

Types of Exception in java.



OOP Interview Questions

What is difference between compiled and an interpreted language

OOP Interview Questions

Difference between final, finally, finalize

Difference between String, StringBuffer, StringBuilder

OOP Interview Questions

- Difference between String, StringBuffer, StringBuilder
- **Explain String Builder.**

StringBuilder is popular in the Java industry because it is mutable, memory-efficient, and faster than String and StringBuffer for repeated string modifications. Since most enterprise applications perform string manipulation in a single thread, StringBuilder provides the best balance of performance and simplicity.

What is System class in java?

The System class is a predefined utility class in the java.lang package that provides access to system resources, standard input/output streams, environment information, time-related methods, and garbage collection requests.

OOP Interview Questions

What is Static Keyword in java?

Static class can have non static members?

Can we create an object of static class?

Explain Multithreading in java

OOP Interview Questions

what is JDBC?

OOP Interview Questions

what is used of encapsulation(explain with used in project scenario) what is used of parent class and child class.

Difference between == and .equals

What is the difference between Object Equality and Reference Equality?

Reference equality (==) checks if two variables point to the exact same memory address — the same object instance.

Object equality (.equals()) checks if two objects are logically equivalent based on their content/state.

```
String a = new String("hello");
String b = new String("hello");

a == b       // false — different objects in heap
a.equals(b)  // true  — same content
```

What is object identity?

- Object identity is what makes an object uniquely itself — its memory address / reference in the JVM.
- Two objects can have identical state (same equality) but still be different identities.

```
String a = new String("hi");
String b = new String("hi");

// same equality    → a.equals(b) = true
// different identity → a == b = false
```

OOP Interview Questions

Do Interface have constructor?

No

Do abstract class have a constructor?

Yes

What is sealed class in java?

- A **sealed class** is a class that **restricts which classes can inherit from it**.
- It was introduced as a standard feature in **Java 17**.

```
sealed class Vehicle
     permits Car, Bike {
}
final class Car extends Vehicle { }
final class Bike extends Vehicle { }
```

What are SOLID principles of OOP?

Darshan UNIVERSITY

- **SOLID** is a set of 5 object-oriented design principles that help you write code that is easier to maintain, extend, test, and understand.

S	**Single Responsibility Principle (SRP)**	A class should do one thing and do it well.
O	**Open/Closed Principle (OCP)**	Software entities should be open for extension but closed for modification. i.e You should be able to add new functionality without changing existing code.
L	**Liskov Substitution Principle (LSP)**	Objects of a superclass should be replaceable with objects of its subclasses without breaking the program.
I	**Interface Segregation Principle (ISP)**	Clients should not be forced to depend on methods they do not use.
D	**Dependency Inversion Principle (DIP)**	High-level modules should not depend on low-level modules. Both should depend on abstractions.

What is the difference between IS-A and HAS-A? OR

IS-A = inheritance. The child class is a specialized type of the parent.

```
class Animal {}
class Dog extends Animal {}   // Dog IS-A Animal
```

HAS-A = composition. A class contains a reference to another class as a field.

```
class Car {
    Engine engine;          // Car HAS-A Engine
}
```

Which should be preferred: IS-A or HAS-A?

- Generally, **HAS-A (composition) is preferred over IS-A (inheritance)** because it provides:
  - Better encapsulation
  - Lower coupling
  - Greater flexibility
  - Easier maintenance

What is object composition vs inheritance? OR Inheritance vs Delegation

- **Inheritance** reuses behavior by extending a class. The subclass gets the parent's implementation automatically but is tightly coupled to it.
- **Composition** reuses behavior by holding an instance of another class and delegating to it. Relationships are defined at runtime and can change.

What is object composition vs inheritance?

Feature	Inheritance	Composition/Delegation
Relationship	IS-A	HAS-A
Keyword	extends	Object reference
Coupling	Tight	Loose
Flexibility	Less	More
Runtime behaviour change	Difficult	Easy
Polymorphism	Directly supported	Through interfaces
Maintenance	Harder in large hierarchies	Easier

What is delegation in OOP?

- Delegation is when an object hands off a task to a helper object instead of doing it itself.
- It's the runtime mechanism that makes composition work.

```
class Printer {
    void print(String text) { System.out.println(text); }
}

class Office {
    Printer printer = new Printer();

    void printReport(String text) {
        printer.print(text);   // delegates to Printer
    }
}
```

Office doesn't implement printing logic — it delegates to Printer.
Responsibilities stay focused.

What is delegation in OOP?

- Delegation is when an object hands off a task to a helper object instead of doing it itself.
- It's the runtime mechanism that makes composition work.

```
class Printer {
    void print(String text) { System.out.println(text); }
}

class Office {
    Printer printer = new Printer();

    void printReport(String text) {
        printer.print(text);   // delegates to Printer
    }
}
```

Office doesn't implement printing logic — it delegates to Printer.
Responsibilities stay focused.

Thank You

Darshan UNIVERSITY

योगः कर्मसु कौशलम्